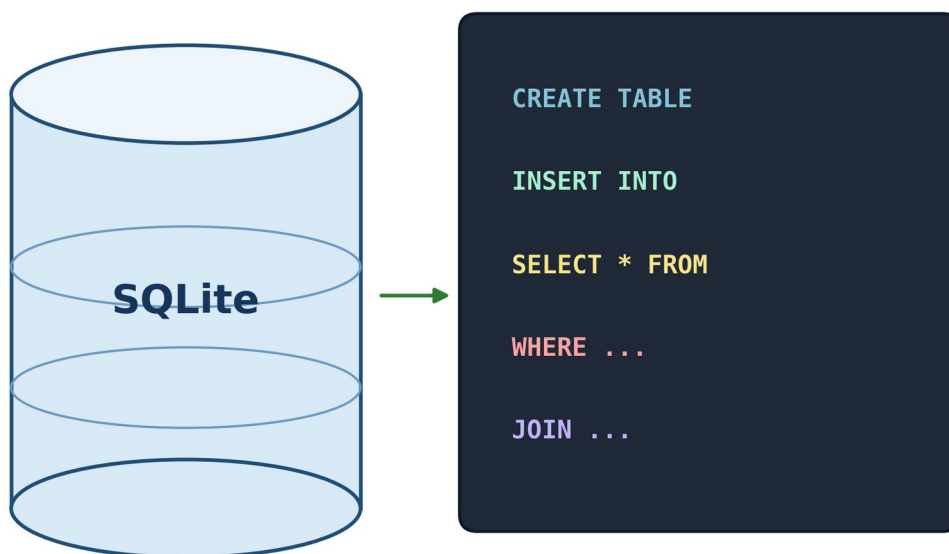


APRENDENDO BANCO DE DADOS COM SQLITE

Fundamentos, modelagem, terminal Linux e prática com SQL

Dados + SQL + prática



Fundamentos, modelagem, terminal, VS Code, exercícios e gabarito

Tutorial passo a passo para alunos iniciantes e monitores

Atualizado em 31 de julho de 2026

Apresentação do material

Este tutorial foi elaborado para iniciantes e utiliza somente Linux. O conteúdo parte dos conceitos essenciais, apresenta modelagem relacional e conduz à criação de um banco de Biblioteca Escolar com o terminal sqlite3 e o Visual Studio Code.

Todo o curso utiliza o mesmo projeto, permitindo que o aluno acompanhe a evolução do banco sem trocar de contexto.

Importante: As respostas das atividades não aparecem junto aos exercícios. O gabarito completo está nos apêndices, para que o estudante tente resolver cada desafio antes de conferir a solução.

Público-alvo

- Alunos sem experiência prévia com banco de dados.
- Monitores de oficinas práticas em laboratórios Linux.
- Estudantes de programação que precisam aprender SQL e persistência.

Objetivos de aprendizagem

- Compreender banco de dados, tabelas, registros, colunas e chaves.
- Modelar entidades, atributos, relacionamentos e cardinalidades.
- Criar e manipular um banco SQLite pelo terminal Linux.
- Praticar CREATE, INSERT, SELECT, UPDATE, DELETE e JOIN.
- Usar arquivos .sql, transações, índices e backup básico.
- Construir e testar o banco de uma Biblioteca Escolar.

Carga horária sugerida

Encontro	Conteúdo	Duração sugerida
1	Ambiente Linux, fundamentos e modelagem	2 horas
2	Criação das tabelas e inserção de dados	2 a 3 horas
3	Consultas, filtros, atualização e exclusão	2 a 3 horas
4	JOINS, agrupamentos e transações	2 a 3 horas
5	Índices, backup, revisão e desafio	2 horas

Como o monitor deve usar o tutorial

1. Teste previamente os comandos no mesmo Linux utilizado pelo laboratório.
2. Digite os exemplos lentamente e explique cada parte.
3. Peça aos alunos que reproduzam cada comando antes de avançar.
4. Reserve pausas para os exercícios e consulte o gabarito apenas depois.
5. Leia as mensagens de erro com a turma antes de corrigir.

Sumário do curso

- 0 Preparação do ambiente
- 1 Fundamentos de banco de dados

APRENDENDO BANCO DE DADOS COM SQLITE

- 2 Modelagem de banco de dados
- 3 Primeiros passos com o SQLite no terminal
- 4 Criando as tabelas da Biblioteca Escolar
- 5 Inserindo e consultando dados
- 6 Filtros, ordenação e pesquisa
- 7 Atualização e exclusão de dados
- 8 Relacionamentos e consultas com JOIN
- 9 Funções de agregação e agrupamentos
- 10 Integridade, chaves estrangeiras e transações
- 11 Índices e plano de consulta
- 12 Arquivos SQL, importação, exportação e backup
- 13 Desafio final e roteiro de avaliação
- Apêndice A** Respostas das atividades
- Apêndice B** Guia rápido de comandos
- Apêndice C** Glossário
- Apêndice D** Referências utilizadas

MÓDULO 0

Preparação do ambiente Linux

Instalação do SQLite e organização do projeto no VS Code.

O SQLite armazena o banco em um único arquivo. Neste curso, todos os comandos serão executados em um terminal Linux, preferencialmente Ubuntu ou Debian, usando o programa sqlite3.

0.1 Instalando as ferramentas

Abra um terminal e instale o SQLite:

```
sudo apt update
sudo apt install -y sqlite3
```

Verifique se o Visual Studio Code já está disponível:

```
code --version
```

Se o comando não existir, instale o VS Code pelo gerenciador de aplicativos da distribuição ou pelo pacote oficial. Não é necessária nenhuma extensão de banco de dados para este curso.

0.2 Criando e abrindo a pasta do curso

```
mkdir -p ~/curso-sqlite
cd ~/curso-sqlite
code .
```

No VS Code, abra o terminal integrado pelo menu Terminal > Novo Terminal. O atalho mais comum no Linux é Ctrl + `, usando a tecla de crase.

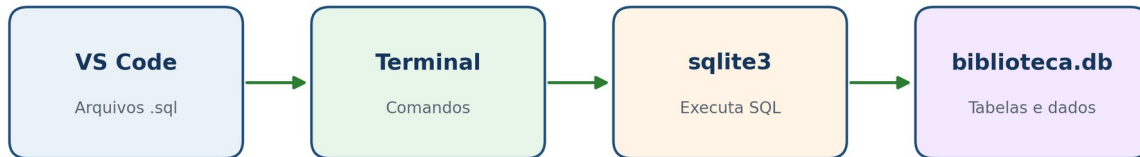
0.3 Verificando o ambiente

```
pwd
sqlite3 --version
```

APRENDENDO BANCO DE DADOS COM SQLITE

O comando `pwd` deve mostrar a pasta `curso-sqlite`. O segundo comando deve exibir a versão do SQLite. Se ambos funcionarem, o ambiente está pronto.

Fluxo de trabalho utilizado no tutorial



O editor organiza os scripts; o terminal chama o programa `sqlite3`; o banco é salvo em um único arquivo.

Figura 1 - Fluxo de trabalho no Linux.

0.4 Checklist do ambiente

- O VS Code abre a pasta `~/curso-sqlite`.
- O terminal integrado está visível.
- `pwd` mostra a pasta correta.
- `sqlite3 --version` apresenta uma versão.
- O aluno sabe que comandos SQL terminam com ponto e vírgula.

ATIVIDADE A1 - Execute `sqlite3 --version` e anote a versão instalada.

ATIVIDADE A2 - Crie a pasta `~/curso-sqlite`, abra-a no VS Code e confirme o caminho atual com `pwd`.

MÓDULO 1

Fundamentos de banco de dados

Entendendo dados, tabelas, registros, colunas e sistemas gerenciadores.

1.1 Dado, informação e banco de dados

Um dado é um valor isolado, como “Ana”, “1º A” ou “2026-07-20”. Quando os dados são organizados e interpretados, tornam-se informação: por exemplo, “Ana, da turma 1º A, pegou um livro emprestado em 20 de julho de 2026”.

Um banco de dados é uma coleção organizada de dados que permite inserir, consultar, alterar e remover informações. Em vez de guardar tudo em arquivos de texto separados, o banco define estruturas e regras que ajudam a manter a consistência.

1.2 O que é um SGBD?

SGBD significa Sistema Gerenciador de Banco de Dados. É o software responsável por armazenar e manipular os dados. SQLite, PostgreSQL, MySQL e SQL Server são exemplos de SGBDs. Neste curso, o SGBD é o SQLite.

SQLite em uma frase: É um banco relacional que normalmente funciona sem servidor separado: a aplicação ou o programa `sqlite3` acessa diretamente um arquivo de banco.

1.3 Tabelas, linhas e colunas

No modelo relacional, os dados são organizados em tabelas. Uma tabela representa um conjunto de itens do mesmo tipo. A tabela alunos, por exemplo, guarda vários alunos.

Anatomia de uma tabela relacional

COLUNA (atributo)



id_aluno	nome	turma	email
1	Ana Souza	1º A	ana.souza@escola.test
2	Bruno Lima	1º A	bruno.lima@escola.test
3	Carla Santos	2º B	carla.santos@escola.test

LINHA (registro)



Figura 2 - Colunas descrevem características; linhas representam registros.

- **Tabela:** estrutura que reúne registros de um mesmo assunto.
- **Coluna ou atributo:** característica armazenada, como nome ou email.
- **Linha ou registro:** um item completo, como um aluno específico.
- **Célula:** valor localizado no encontro de uma linha com uma coluna.
- **Esquema:** definição das tabelas, colunas, tipos e regras do banco.

1.4 Chaves

Uma chave primária identifica cada linha de forma única. Na tabela alunos, id_aluno será a chave primária. Mesmo que duas pessoas tenham o mesmo nome, os identificadores devem ser diferentes.

Uma chave estrangeira guarda o identificador de uma linha existente em outra tabela. Ela cria e protege relacionamentos. Na tabela livros, id_autor informa qual autor escreveu o livro.

1.5 O que é SQL?

SQL é a linguagem usada para definir estruturas e trabalhar com dados relacionais. Os comandos mais importantes para iniciantes são CREATE TABLE, INSERT, SELECT, UPDATE e DELETE.

Comando	Finalidade	Exemplo de ação
CREATE TABLE	Criar uma tabela	Criar a tabela alunos
INSERT	Inserir registros	Cadastrar um aluno
SELECT	Consultar dados	Listar os alunos
UPDATE	Alterar registros	Corrigir um email
DELETE	Remover registros	Excluir um registro de teste

ATIVIDADE A3 - Explique com suas palavras a diferença entre banco de dados e SGBD.

ATIVIDADE A4 - Na tabela alunos da Figura 2, identifique uma coluna, uma linha e uma célula.

MÓDULO 2

Modelagem de banco de dados

Transformando uma situação do mundo real em tabelas relacionadas.

2.1 Antes do SQL: compreender o problema

Modelar um banco significa decidir quais informações precisam ser guardadas e como elas se relacionam. O primeiro passo não é escrever código: é entender os requisitos.

Cenário do curso: Uma biblioteca escolar precisa cadastrar autores, livros e alunos. Também precisa registrar quem pegou cada livro, a data do empréstimo, a previsão de devolução e a situação atual.

2.2 Entidades e atributos

Entidade é algo sobre o qual o sistema precisa guardar dados. No cenário, as entidades principais são Autor, Livro, Aluno e Empréstimo.

Entidade	Atributos principais	Identificador
Autor	nome, nacionalidade	id_autor
Livro	título, ISBN, ano, categoria, quantidade	id_livro
Aluno	nome, turma, email	id_aluno
Empréstimo	datas e status	id_emprestimo

2.3 Relacionamentos e cardinalidade

Cardinalidade indica quantas ocorrências de uma entidade podem se relacionar com outra. Para o projeto:

- Um autor pode escrever vários livros; cada livro possui um autor no modelo simplificado: 1 para N.
- Um aluno pode realizar vários empréstimos; cada empréstimo pertence a um aluno: 1 para N.
- Um livro pode aparecer em vários empréstimos ao longo do tempo; cada empréstimo se refere a um livro: 1 para N.
- Aluno e Livro formam conceitualmente uma relação N para N ao longo do tempo. A entidade Empréstimo resolve essa relação e guarda dados adicionais.

Modelo lógico simplificado - Biblioteca Escolar

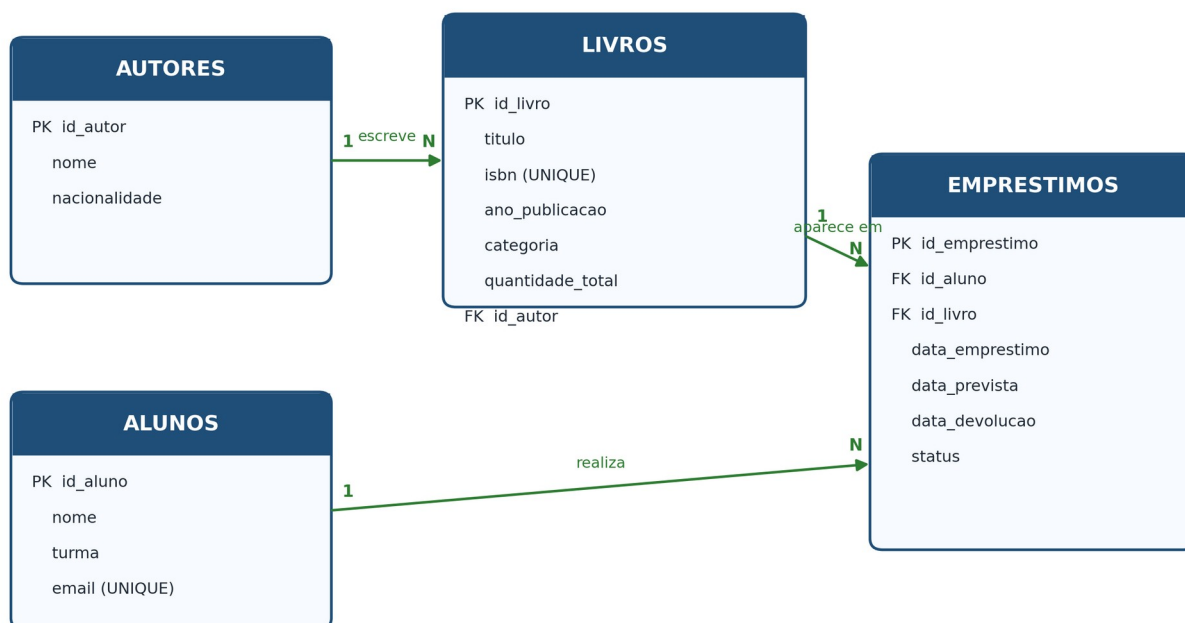


Figura 3 - Modelo lógico simplificado da Biblioteca Escolar.

2.4 Regras de integridade

- Todo livro deve ter título, ISBN, categoria e autor.
- O ISBN não pode se repetir.
- A quantidade total não pode ser negativa.
- Todo empréstimo deve apontar para aluno e livro existentes.
- O status deve ser ABERTO, DEVOLVIDO ou ATRASADO.
- As datas serão guardadas no formato ISO: AAAA-MM-DD.

Datas no SQLite: SQLite não possui uma classe de armazenamento DATE exclusiva. Neste projeto, datas serão armazenadas como TEXT no formato AAAA-MM-DD, que facilita ordenação e uso das funções de data.

2.5 Normalização básica

Normalização é um conjunto de ideias para reduzir repetição e inconsistência. Neste nível introdutório, use três perguntas:

1. Cada célula guarda apenas um valor simples? Evite colocar vários telefones ou vários autores na mesma célula.
2. Os dados de um assunto estão na tabela correta? O nome do autor pertence a autores, não deve ser repetido em cada livro.
3. A tabela guarda fatos sobre uma única entidade? Empréstimos deve conter dados do empréstimo e referências, não copiar todos os dados do aluno e do livro.

ATIVIDADE A6 - Identifique as quatro entidades principais do cenário da biblioteca.

ATIVIDADE A7 - Proponha três atributos para a entidade Livro e explique o significado de cada um.

ATIVIDADE A8 - Qual é a cardinalidade entre Autor e Livro? Explique.

ATIVIDADE A9 - Por que não é recomendável escrever o nome completo do autor diretamente em todas as linhas da tabela livros?

ATIVIDADE A10 - Explique como a entidade Empréstimo resolve a relação entre Aluno e Livro.

MÓDULO 3

Primeiros passos com o SQLite no terminal

Criando um arquivo de banco e conhecendo os comandos do shell.

3.1 Criando o primeiro banco

No terminal integrado, dentro da pasta curso-sqlite, execute:

```
sqlite3 biblioteca.db
```

Se o arquivo não existir, ele será criado. O terminal mudará para um prompt semelhante a `sqlite>`. A partir desse momento, comandos SQL terminados por ponto e vírgula serão executados no banco aberto.

3.2 Configurando a visualização

```
.headers on
.mode column
.nullvalue NULL
```

- **.headers on:** mostra o nome das colunas nos resultados.
- **.mode column:** organiza a saída em colunas legíveis.
- **.nullvalue NULL:** mostra explicitamente quando um valor é nulo.

Comandos com ponto: Comandos iniciados por ponto, como `.tables`, pertencem ao programa `sqlite3`. Eles não são SQL e normalmente não recebem ponto e vírgula.

3.3 Comandos essenciais do CLI

Comando	O que faz
<code>.help</code>	Mostra ajuda dos comandos do terminal.
<code>.databases</code>	Lista os bancos abertos e seus arquivos.
<code>.tables</code>	Lista as tabelas existentes.
<code>.schema</code>	Mostra o SQL usado para criar as tabelas.
<code>.schema alunos</code>	Mostra apenas a estrutura da tabela alunos.
<code>.read arquivo.sql</code>	Executa comandos armazenados em um arquivo.
<code>.dump</code>	Gera uma representação textual do banco.
<code>.quit</code>	Encerra o programa <code>sqlite3</code> .

3.4 Teste rápido

```
CREATE TABLE teste (
    id INTEGER PRIMARY KEY,
    mensagem TEXT NOT NULL
);

INSERT INTO teste (mensagem)
VALUES ('Olá, SQLite!');

SELECT * FROM teste;
```

O resultado deve mostrar uma linha. Depois, remova a tabela de teste:

```
DROP TABLE teste;
```

3.5 Encerrando corretamente

```
.quit
```

O arquivo biblioteca.db continua na pasta. Ao executar sqlite3 biblioteca.db novamente, o mesmo banco será reaberto.

ATIVIDADE A11 - Abra biblioteca.db, ative cabeçalhos, use o modo column e execute .databases.

ATIVIDADE A12 - Crie a tabela teste, insira duas mensagens, consulte os dados e depois remova a tabela.

ATIVIDADE A13 - Explique a diferença entre SELECT * FROM teste; e .tables.

MÓDULO 4

Criando as tabelas da Biblioteca Escolar

DDL, tipos, chaves e restrições no SQLite.

4.1 Organizando o script no VS Code

Crie um arquivo chamado 01_criar_estrutura.sql. Escrever comandos em arquivo é melhor do que depender apenas do histórico do terminal: o script pode ser salvo, revisado, corrigido e executado novamente.

4.2 Ativando chaves estrangeiras

```
PRAGMA foreign_keys = ON;
```

No SQLite, a fiscalização de chaves estrangeiras deve ser ativada para cada conexão. Por isso, este comando aparece no início dos scripts e deve ser executado ao abrir uma nova sessão.

4.3 Tipos de dados básicos

SQLite utiliza tipagem dinâmica e trabalha com cinco classes de armazenamento principais. Os tipos declarados nas colunas orientam a afinidade de tipo, mas tabelas comuns são flexíveis. Para iniciantes, declare tipos coerentes e use restrições para expressar regras.

Classe	Uso comum	Exemplos
--------	-----------	----------

NULL	Ausência de valor	email não informado
INTEGER	Números inteiros	id, quantidade, ano
REAL	Números com parte decimal	preço, média
TEXT	Texto e datas em formato ISO	nome, 2026-07-31
BLOB	Conteúdo binário	imagem ou arquivo

4.4 Criando autores

```
CREATE TABLE autores (
  id_autor INTEGER PRIMARY KEY,
  nome TEXT NOT NULL,
  nacionalidade TEXT
);
```

- **INTEGER PRIMARY KEY:** identificador inteiro único da linha.
- **NOT NULL:** o campo nome é obrigatório.
- **nacionalidade:** pode receber NULL porque não é obrigatória.

4.5 Criando alunos

```
CREATE TABLE alunos (
  id_aluno INTEGER PRIMARY KEY,
  nome TEXT NOT NULL,
  turma TEXT NOT NULL,
  email TEXT UNIQUE
);
```

UNIQUE impede que dois alunos tenham o mesmo email, mas permite valores NULL. Esta é uma regra simplificada para fins didáticos.

4.6 Criando livros

```
CREATE TABLE livros (
  id_livro INTEGER PRIMARY KEY,
  titulo TEXT NOT NULL,
  isbn TEXT NOT NULL UNIQUE,
  ano_publicacao INTEGER
  CHECK (ano_publicacao BETWEEN 1400 AND 2100),
  categoria TEXT NOT NULL,
  quantidade_total INTEGER NOT NULL DEFAULT 1
  CHECK (quantidade_total >= 0),
  id_autor INTEGER NOT NULL,
  FOREIGN KEY (id_autor)
  REFERENCES autores(id_autor)
  ON UPDATE CASCADE
  ON DELETE RESTRICT
);
```

- **CHECK:** verifica uma condição antes de aceitar a linha.
- **DEFAULT 1:** usa 1 quando a quantidade não for informada.
- **FOREIGN KEY:** exige um autor existente.
- **ON DELETE RESTRICT:** impede excluir um autor que ainda possui livros.
- **ON UPDATE CASCADE:** propaga uma eventual alteração do identificador.

4.7 Criando empréstimos

```
CREATE TABLE empréstimos (
    id_emprestimo INTEGER PRIMARY KEY,
    id_aluno INTEGER NOT NULL,
    id_livro INTEGER NOT NULL,
    data_emprestimo TEXT NOT NULL,
    data_prevista TEXT NOT NULL,
    data_devolucao TEXT,
    status TEXT NOT NULL DEFAULT 'ABERTO'
    CHECK (status IN ('ABERTO', 'DEVOLVIDO', 'ATRASADO')),
    CHECK (data_prevista >= data_emprestimo),
    FOREIGN KEY (id_aluno) REFERENCES alunos(id_aluno),
    FOREIGN KEY (id_livro) REFERENCES livros(id_livro)
);
```

4.8 Executando o arquivo

Salve o arquivo e abra o banco:

```
sqlite3 biblioteca.db
```

Dentro do sqlite3, execute:

```
.read 01_criar_estrutura.sql
.tables
.schema
```

Repetição segura: O arquivo fornecido no pacote remove as tabelas antigas na ordem correta antes de recriá-las. Isso permite reiniciar o laboratório. Não use DROP TABLE em bancos reais sem entender o impacto.

ATIVIDADE A14 - Liste nome e turma de todos os alunos em ordem alfabética. Esta atividade será executada depois da inserção dos dados; apenas escreva a consulta em 04_exercicios.sql.

MÓDULO 5

Inserindo e consultando dados

Primeiros comandos INSERT e SELECT.

5.1 Inserindo uma linha

```
INSERT INTO autores (nome, nacionalidade)
VALUES ('Machado de Assis', 'Brasileira');
```

Os nomes das colunas aparecem entre parênteses. Os valores seguem a mesma ordem. Textos usam aspas simples.

5.2 Inserindo várias linhas

```
INSERT INTO autores (nome, nacionalidade) VALUES
('Clarice Lispector', 'Brasileira'),
('George Orwell', 'Britânica'),
('Antoine de Saint-Exupéry', 'Francesa');
```

5.3 Consultando tudo

```
SELECT * FROM autores;
```

O asterisco significa todas as colunas. Em projetos maiores, prefira indicar as colunas necessárias.

5.4 Consultando colunas específicas

```
SELECT nome, nacionalidade  
FROM autores;
```

5.5 Criando o conjunto inicial

No pacote prático, o arquivo 02_inserir_dados.sql cadastra autores, alunos, livros e empréstimos. Recrie o banco e carregue os dados com:

```
sqlite3 biblioteca.db  
.read 01_criar_estrutura.sql  
.read 02_inserir_dados.sql  
.headers on  
.mode column
```

5.6 Verificando quantidades

```
SELECT COUNT(*) AS total_autores FROM autores;  
SELECT COUNT(*) AS total_alunos FROM alunos;  
SELECT COUNT(*) AS total_livros FROM livros;  
SELECT COUNT(*) AS total_emprestimos FROM emprestimos;
```

O conjunto inicial possui 5 autores, 5 alunos, 8 livros e 7 empréstimos.

ATIVIDADE A15 - Liste título e ano dos livros publicados a partir de 2000.

ATIVIDADE A16 - Liste os títulos que contêm a letra “a”, sem diferenciar maiúsculas de minúsculas.

ATIVIDADE A17 - Mostre os três livros mais antigos do banco.

ATIVIDADE A18 - Cadastre a autora Lygia Fagundes Telles, de nacionalidade brasileira.

ATIVIDADE A19 - Cadastre o livro As Meninas, publicado em 1973, relacionando-o à autora criada na atividade anterior.

Dica: Você pode descobrir o id da autora com uma consulta *SELECT*.

MÓDULO 6

Filtros, ordenação e pesquisa

WHERE, operadores, LIKE, IN, BETWEEN, ORDER BY e LIMIT.

6.1 Filtrando com WHERE

```
SELECT titulo, categoria  
FROM livros  
WHERE categoria = 'Romance';
```

6.2 Operadores de comparação

Operador	Significado	Exemplo
=	igual	ano_publicacao = 1949
<> ou !=	diferente	status <> 'DEVOLVIDO'
>	maior que	quantidade_total > 2
>=	maior ou igual	ano_publicacao >= 2000
<	menor que	ano_publicacao < 1950
<=	menor ou igual	quantidade_total <= 3

6.3 Combinando condições

```
SELECT titulo, ano_publicacao
FROM livros
WHERE categoria = 'Romance'
AND ano_publicacao >= 1900;
```

```
SELECT titulo, categoria
FROM livros
WHERE categoria = 'Ficção'
OR categoria = 'Contos';
```

6.4 Pesquisando partes de texto com LIKE

```
SELECT titulo
FROM livros
WHERE titulo LIKE '%Príncipe%';
```

- %: representa zero ou mais caracteres.
- _: representa exatamente um caractere.
- '%cão%': encontra textos que contêm “cão”.

6.5 IN e BETWEEN

```
SELECT titulo, categoria
FROM livros
WHERE categoria IN ('Romance', 'Ficção');
```

```
SELECT titulo, ano_publicacao
FROM livros
WHERE ano_publicacao BETWEEN 1940 AND 2000;
```

6.6 Ordenando e limitando

```
SELECT titulo, ano_publicacao
FROM livros
ORDER BY ano_publicacao DESC, titulo ASC
LIMIT 5;
```

Boa prática: Sem ORDER BY, não confie na ordem em que as linhas aparecem. Quando a ordem importa, declare-a explicitamente.

MÓDULO 7

Atualização e exclusão de dados

Alterando registros com cuidado e evitando mudanças acidentais.

7.1 Atualizando com UPDATE

```
UPDATE livros
SET quantidade_total = 5
WHERE titulo = 'Dom Casmurro';
```

O WHERE escolhe quais linhas serão alteradas. Antes de executar UPDATE, é recomendável testar a mesma condição com SELECT.

Consulta de conferência antes do UPDATE

```
SELECT *
FROM livros
WHERE titulo = 'Dom Casmurro';
```

7.2 Atualizando várias colunas

```
UPDATE alunos
SET turma = '2º A',
    email = 'novo.email@escola.test'
WHERE id_aluno = 1;
```

7.3 Excluindo com DELETE

```
DELETE FROM alunos
WHERE email = 'teste@escola.test';
```

Perigo: UPDATE ou DELETE sem WHERE pode alterar ou remover todas as linhas da tabela. Em uma aula, destaque visualmente o WHERE antes de executar.

7.4 Conferindo o resultado

```
SELECT changes() AS linhas_afetadas;
```

A função changes() informa quantas linhas foram modificadas pelo último INSERT, UPDATE ou DELETE da conexão.

ATIVIDADE A20 - Altere a quantidade total de Dom Casmurro para 5. Consulte antes e depois da alteração.

ATIVIDADE A21 - Crie um aluno chamado Aluno de Teste e depois exclua somente esse registro usando uma condição segura.

MÓDULO 8

Relacionamentos e consultas com JOIN

Reunindo dados que estão em tabelas diferentes.

8.1 Por que usar JOIN?

A tabela livros guarda id_autor, não repete o nome do autor. Para mostrar o título e o autor na mesma consulta, relacionamos as tabelas usando JOIN.

8.2 INNER JOIN

```
SELECT
    l.titulo,
    a.nome AS autor
FROM livros AS l
INNER JOIN autores AS a
    ON a.id_autor = l.id_autor
ORDER BY l.titulo;
```

- **I e a:** apelidos que encurtam os nomes das tabelas.
- **ON:** define a condição que liga as linhas.
- **AS autor:** define um nome amigável para a coluna exibida.
- **INNER JOIN:** retorna apenas combinações que possuem correspondência.

8.3 Consulta com três tabelas

```
SELECT
    e.id_emprestimo,
    al.nome AS aluno,
    l.titulo AS livro,
    e.data_prevista,
    e.status
FROM empréstimos AS e
INNER JOIN alunos AS al
    ON al.id_aluno = e.id_aluno
INNER JOIN livros AS l
    ON l.id_livro = e.id_livro
ORDER BY e.data_prevista;
```

8.4 LEFT JOIN

LEFT JOIN preserva todas as linhas da tabela à esquerda, mesmo quando não existe correspondência à direita. Isso é útil para localizar alunos sem empréstimos.

```
SELECT al.id_aluno, al.nome
FROM alunos AS al
LEFT JOIN empréstimos AS e
    ON e.id_aluno = al.id_aluno
WHERE e.id_emprestimo IS NULL;
```

ATIVIDADE A22 - Liste cada livro junto com o nome do autor, ordenando pelo título.

ATIVIDADE A23 - Liste empréstimos com status ABERTO ou ATRASADO, mostrando aluno, livro, data prevista e status.

ATIVIDADE A26 - Mostre os alunos que nunca fizeram empréstimo usando LEFT JOIN.

MÓDULO 9

Funções de agregação e agrupamentos

Resumindo dados com COUNT, SUM, AVG, MIN, MAX, GROUP BY e HAVING.

9.1 Funções de agregação

Função	Resultado
COUNT(*)	Quantidade de linhas.
SUM(coluna)	Soma dos valores.
AVG(coluna)	Média dos valores.
MIN(coluna)	Menor valor.
MAX(coluna)	Maior valor.

```
SELECT
    COUNT(*) AS total_titulos,
    SUM(quantidade_total) AS total_exemplares,
    AVG(ano_publicacao) AS media_ano
FROM livros;
```

9.2 Agrupando com GROUP BY

```
SELECT
    categoria,
    COUNT(*) AS quantidade_de_titulos
FROM livros
GROUP BY categoria
ORDER BY quantidade_de_titulos DESC;
```

GROUP BY forma grupos com valores iguais. COUNT(*) é calculado separadamente para cada categoria.

9.3 Filtrando grupos com HAVING

```
SELECT categoria, COUNT(*) AS total
FROM livros
GROUP BY categoria
HAVING COUNT(*) >= 2;
```

WHERE filtra linhas antes do agrupamento. HAVING filtra os grupos depois do GROUP BY.

9.4 Contando relacionamentos

```
SELECT
    al.nome,
    COUNT(e.id_emprestimo) AS total_emprestimos
FROM alunos AS al
LEFT JOIN emprestimos AS e
    ON e.id_aluno = al.id_aluno
GROUP BY al.id_aluno, al.nome
ORDER BY total_emprestimos DESC, al.nome;
```

ATIVIDADE A24 - Mostre quantos títulos existem em cada categoria.

ATIVIDADE A25 - Mostre somente as categorias com dois ou mais títulos.

ATIVIDADE A27 - Mostre a quantidade de empréstimos para cada status.

Integridade, chaves estrangeiras e transações

Protegendo os dados e agrupando operações que devem acontecer juntas.

10.1 Verificando chaves estrangeiras

```
PRAGMA foreign_keys;
```

O resultado 1 indica fiscalização ativa. Se retornar 0, execute `PRAGMA foreign_keys = ON;` fora de uma transação.

```
PRAGMA foreign_key_check;
```

Se não aparecer nenhuma linha, não foram encontradas violações de chave estrangeira.

10.2 Tentando inserir uma referência inexistente

```
INSERT INTO livros (
    titulo, isbn, categoria, quantidade_total, id_autor
)
VALUES (
    'Livro inválido', '000', 'Teste', 1, 9999
);
```

Com `foreign_keys` ativado, o comando falha porque não existe autor com id 9999. Esse erro demonstra a proteção do relacionamento.

10.3 O que é uma transação?

Uma transação agrupa operações para que sejam confirmadas juntas ou desfeitas juntas. SQLite executa acessos dentro de transações automaticamente, mas podemos abrir uma transação explícita com `BEGIN`.

```
BEGIN TRANSACTION;

INSERT INTO alunos (nome, turma, email)
VALUES ('Teste de Transação', 'TESTE', 'transacao@escola.test');

SELECT *
FROM alunos
WHERE email = 'transacao@escola.test';

ROLLBACK;
```

`ROLLBACK` desfaz o `INSERT`. Se o objetivo fosse manter a alteração, usaríamos `COMMIT`.

10.4 Exemplo de empréstimo atômico

```
BEGIN TRANSACTION;

INSERT INTO emprestimos (
    id_aluno, id_livro,
    data_emprestimo, data_prevista, status
)
VALUES (2, 3, '2026-08-01', '2026-08-08', 'ABERTO');

UPDATE livros
SET quantidade_total = quantidade_total - 1
WHERE id_livro = 3
    AND quantidade_total > 0;

COMMIT;
```

Simplificação didática: Em um sistema real, `quantidade_total` e `disponibilidade` são conceitos diferentes. Seria melhor modelar exemplares físicos ou calcular empréstimos ativos. O exemplo serve apenas para praticar transações.

ATIVIDADE A30 - Inicie uma transação, insira um aluno de teste, consulte a linha, execute `ROLLBACK` e confirme que a linha desapareceu.

ATIVIDADE A32 - Execute `PRAGMA foreign_key_check;` e interprete o resultado.

MÓDULO 11

Índices e plano de consulta

Uma introdução ao desempenho sem perder o foco nos fundamentos.

11.1 O que é um índice?

Um índice é uma estrutura auxiliar que pode acelerar buscas e ordenações. A ideia é semelhante ao índice de um livro: localizar um tema pelo índice costuma ser mais rápido do que ler todas as páginas.

Índices também ocupam espaço e precisam ser atualizados quando dados mudam. Por isso, não devemos criar índices em todas as colunas sem motivo.

11.2 Criando um índice

```
CREATE INDEX IF NOT EXISTS idx_alunos_nome
ON alunos(nome);
```

11.3 Observando o plano

```
EXPLAIN QUERY PLAN
SELECT *
FROM alunos
WHERE nome = 'Ana Souza';
```

A saída pode mencionar `SEARCH` e o nome do índice. Em uma tabela pequena, a diferença de tempo não será perceptível, mas o comando ensina como o banco planeja a consulta.

11.4 Índices já existentes no projeto

```
CREATE INDEX idx_livros_titulo
ON livros(titulo);

CREATE INDEX idx_emprestimos_aluno
ON empréstimos(id_aluno);

CREATE INDEX idx_emprestimos_livro
ON empréstimos(id_livro);
```

ATIVIDADE A28 - Crie o índice `idx_alunos_nome` para a coluna `nome` da tabela `alunos`.

ATIVIDADE A29 - Use `EXPLAIN QUERY PLAN` em uma consulta que pesquise um aluno pelo nome. Anote se o plano menciona o índice.

Arquivos SQL e backup

Executando scripts e criando cópias de segurança no Linux.

12.1 Executando um arquivo .sql

```
.read 03_consultas_exemplos.sql
```

O comando `.read` executa o conteúdo do arquivo dentro do terminal `sqlite3`.

12.2 Executando pelo terminal Linux

```
sqlite3 biblioteca.db ".read 03_consultas_exemplos.sql"
```

Esse formato executa o script sem entrar no modo interativo.

12.3 Backup textual com .dump

```
.output backup_biblioteca.sql  
.dump  
.output stdout
```

Para restaurar o backup em outro arquivo:

```
sqlite3 biblioteca_restaurada.db ".read backup_biblioteca.sql"
```

12.4 Cópia simples do banco

Com o `sqlite3` fechado, o arquivo do banco também pode ser copiado no Linux:

```
cp biblioteca.db biblioteca_backup.db
```

Boa prática: Para a atividade de laboratório, feche o banco antes de copiar. Em sistemas reais com gravações simultâneas, utilize os mecanismos próprios de backup do SQLite.

ATIVIDADE A31 - Gere `backup_biblioteca.sql` com `.dump` e confirme o arquivo usando `ls -lh`.

Desafio final e roteiro de avaliação

Consolidando modelagem, criação, CRUD, consultas e integridade.

13.1 Desafio: ampliar a biblioteca

Crie uma nova versão do projeto com a entidade editoras. Cada livro deve referenciar uma editora. A atividade pode ser feita em dupla.

1. Escreva três requisitos para editoras.
2. Desenhe a entidade Editora com `id_editora`, nome e cidade.
3. Defina a cardinalidade entre Editora e Livro.
4. Crie a tabela editoras.

APRENDENDO BANCO DE DADOS COM SQLITE

5. Adicione id_editora à tabela livros ou recrie o banco com o novo modelo.
6. Insira pelo menos três editoras.
7. Associe os livros às editoras.
8. Escreva uma consulta que mostre título, autor e editora.
9. Escreva uma consulta que conte livros por editora.
10. Gere um backup textual da versão final.

13.2 Critérios de avaliação

Critério	Evidência esperada	Peso sugerido
Modelagem	Entidades, atributos, chaves e cardinalidades coerentes	25%
Estrutura SQL	Tabelas e restrições executam sem erro	20%
Manipulação	INSERT, UPDATE e DELETE corretos e seguros	15%
Consultas	Filtros, JOIN e agregações corretas	25%
Organização	Arquivos .sql legíveis e banco testado	10%
Explicação	Aluno consegue explicar suas escolhas	5%

13.3 Checklist final do aluno

- Consigo explicar tabela, linha, coluna, chave primária e chave estrangeira.
- Consigo abrir e fechar um banco pelo terminal.
- Consigo criar tabelas com regras básicas.
- Consigo inserir, consultar, atualizar e excluir registros.
- Consigo usar WHERE, LIKE, ORDER BY e LIMIT.
- Consigo relacionar tabelas com INNER JOIN e LEFT JOIN.
- Consigo resumir dados com COUNT e GROUP BY.
- Entendo a finalidade de uma transação e de um índice.
- Consigo executar um arquivo .sql e gerar um backup com .dump.

APÊNDICE B - Guia rápido de comandos

Objetivo	Comando
Ver versão	sqlite3 --version
Abrir/criar banco	sqlite3 biblioteca.db
Sair	.quit
Ajuda	.help
Listar tabelas	.tables
Ver esquema	.schema
Executar arquivo	.read arquivo.sql
Mostrar cabeçalhos	.headers on
Formato em colunas	.mode column
Ativar FK	PRAGMA foreign_keys = ON;
Verificar FK	PRAGMA foreign_key_check;
Consultar	SELECT ... FROM ...;
Inserir	INSERT INTO ... VALUES (...);
Atualizar	UPDATE ... SET ... WHERE ...;
Excluir	DELETE FROM ... WHERE ...;
Iniciar transação	BEGIN TRANSACTION;
Confirmar	COMMIT;
Desfazer	ROLLBACK;
Backup textual	.dump

APÊNDICE C - Glossário

Atributo: Característica representada por uma coluna.

Banco de dados: Coleção organizada de dados.

Cardinalidade: Quantidade de ocorrências que podem se relacionar.

Chave estrangeira: Coluna que referencia a chave de outra tabela.

Chave primária: Identificador único de uma linha.

CLI: Interface de linha de comando.

DDL: Comandos de definição, como CREATE TABLE e DROP TABLE.

DML: Comandos de manipulação, como INSERT, UPDATE e DELETE.

Entidade: Objeto ou conceito sobre o qual guardamos dados.

Esquema: Estrutura formal das tabelas e restrições.

Índice: Estrutura auxiliar que pode acelerar consultas.

JOIN: Operação que combina linhas de tabelas relacionadas.

NULL: Ausência de valor conhecido ou informado.

Registro: Uma linha de uma tabela.

Relacionamento: Associação entre entidades.

Restrição: Regra que limita os valores aceitos.

SGBD: Software que gerencia bancos de dados.

SQL: Linguagem para definição e manipulação de bancos relacionais.

Transação: Grupo de operações confirmado ou desfeito como unidade.